

Translated

From: e-maxx.ru

Arbitrary-Precision Arithmetic

Arbitrary-Precision arithmetic, also known as "bignum" or simply "long arithmetic" is a set of data structures and algorithms which allows to process much greater numbers than can be fit in standard data types. Here are several types of arbitrary-precision arithmetic.

Classical Integer Long Arithmetic

The main idea is that the number is stored as an array of its "digits" in some base. Several most frequently used bases are decimal, powers of decimal (10^4 or 10^9) and binary.

Operations on numbers in this form are performed using "school" algorithms of column addition, subtraction, multiplication and division. It's also possible to use fast multiplication algorithms: fast Fourier transform and Karatsuba algorithm.

Here we describe long arithmetic for only non-negative integers. To extend the algorithms to handle negative integers one has to introduce and maintain additional "negative number" flag or use two's complement integer representation.

Data Structure

We'll store numbers as a `vector<int>`, in which each element is a single "digit" of the number.

```
typedef vector<int> lnum;
```

To improve performance we'll use 10^9 as the base, so that each "digit" of the long number contains 9 decimal digits at once.

```
const int base = 1000*1000*1000;
```

Digits will be stored in order from least to most significant. All operations will be implemented so that after each of them the result doesn't have any leading zeros, as long as operands didn't have any leading zeros either. All operations which might result in a number with leading zeros should be followed by code which removes them. Note that in this representation there are two valid notations for number zero: an empty vector, and a vector with a single zero digit.